

Invertible Normalizing Flows for Generative Image Modeling: Executable Methods and Results Log

Matt Tivnan*

August 31, 2026

Abstract

This document is the executable methods-and-results log for the invertible normalizing flow project. Each numbered step of the pipeline is one `step*.py` script paired with one YAML config in `configs/` and one output folder under `outputs/`; the tables and figures below are included directly from those output folders, so the compiled PDF is an exact record of what was run and what it produced. Step 1 (this version) builds PyTorch datasets and dataloaders with deterministic train/val/test splits for the standard generative-image-modeling benchmarks: MNIST, FashionMNIST, CIFAR-10/100, SVHN, the MedMNIST 2D collection, CelebA, ImageNet, AFHQ, FFHQ, CelebA-HQ, and LSUN bedroom.

1 Execution environment

- Host: `axis03`; container: `recon-dev` (image `localhost:5000/recon:acr-20260819-v1`; PyTorch 2.11.0+cu128, torchvision 0.26.0, CUDA 12.4, 2 GPUs).
- Repository mounted at `/dev_ws/invertible_normalizing_flow_20260831` inside the container (shared workspace `/home/staticct/recon-dev/workspace` on the host).
- Datasets live on the `axis03` data drive at `/data/image_benchmarks`, mounted into the container at the same path. This is a shared local repository of public benchmarks, reused across projects; nothing dataset-sized is stored in git.
- All steps are run as `docker exec recon-dev python /dev_ws/invertible_normalizing_flow_20260831/s`
- Exact package versions, the resolved config, and the command line for the run shown in this PDF are recorded in `outputs/step1_datasets/data/provenance.json`.

2 Step 1: Datasets and dataloaders

2.1 Methods

- Code: `step1_datasets.py` (repository root). Config: `configs/step1_datasets.yml`. Outputs: `outputs/step1_datasets/{data,figures}/`.
- Command executed for this document: `python step1_datasets.py` (runs every dataset enabled in the config).

*Repository: github.com/tivnanmatt/invertible_normalizing_flow_20260831

- The script exposes a library API used by later steps: `build_dataset(name, cfg)` returns a bundle of three `torch.utils.data.Dataset` splits plus metadata, and `get_dataloaders(bundle, cfg)` wraps them in `DataLoaders` (batch size, workers, pinning from the config).
- Auto-download datasets: MNIST, FashionMNIST, CIFAR-10, CIFAR-100, SVHN (torchvision); the MedMNIST 2D collection (`medmnist` package, official train/val/test); Imagenette (fast.ai 10-class ImageNet subset, used as a fast stand-in for ImageNet); AFHQ (StarGAN-v2 release, fetched from the official Dropbox link); CelebA (torchvision Google-Drive download, quota permitting).
- Manual-download datasets (access terms or hosting): ImageNet (ILSVRC2012), FFHQ, CelebA-HQ, LSUN bedroom. When absent, the script records them as *unavailable* together with exact acquisition instructions (Section 2.4) instead of failing.
- Train/val/test policy (deterministic, seeded from the config): official splits are used whenever they exist (MedMNIST, CelebA); datasets with only train/test get a validation set carved from train (10%); datasets whose public split is train/val only (Imagenette, ImageNet, AFHQ) use the official val as *test* and carve val from train; single-pool datasets (FFHQ, CelebA-HQ) are split 80/10/10.
- Transforms: `ToTensor` to $[0, 1]$ floats only; variable-size natural-image datasets additionally get `Resize+CenterCrop` to the per-dataset `image_size` in the config. No normalization or dequantization is baked in here – for flow training those belong to the model step so the data step stays lossless.
- Per dataset the run itself is the test: all three splits are instantiated, real batches are pulled through the dataloaders, per-channel mean/std are computed over up to 20 train batches, and three figures are written (sample grid per split, pixel-intensity histogram, class distribution where integer labels exist).
- Machine-readable outputs: one JSON per dataset, `summary.json`, `summary.csv`, `provenance.json`, and the L^AT_EX fragments (`summary_table.tex`, `figures_step1.tex`, `unavailable_step1.tex`) that are included verbatim below.

2.2 Configuration (verbatim)

```
# Config for step1_datasets.py -- datasets & dataloaders.
#
# data_root is the shared benchmark-dataset repository on the axis03 data
# drive, mounted into the recon-dev container at the same path
# (/data/image_benchmarks). Datasets download/live there once and are reused
# across projects; nothing dataset-sized lands in the git repo.

seed: 20260831
data_root: /data/image_benchmarks
output_root: outputs/step1_datasets

loader:
  batch_size: 64
  num_workers: 4
  pin_memory: true
```

```

split:
  # carved from the official train pool when no official val split exists
  val_fraction: 0.10
  # only used for single-pool datasets with no official splits (ffhq, celeba_hq)
  test_fraction: 0.10

figures:
  samples_per_split: 8      # images per row in the sample-grid figure
  stats_max_batches: 20    # batches used for per-channel mean/std

datasets:
  # ---- small standard benchmarks (auto-download) ----
  mnist:          {enabled: true}
  fashion_mnist: {enabled: true}
  cifar10:         {enabled: true}
  cifar100:        {enabled: true}
  svhn:           {enabled: true}

  # ---- MedMNIST 2D collection (auto-download) ----
  # All 12 2D members are supported; the run set below spans the modalities
  # (pathology, blood smear, dermoscopy, X-ray, ultrasound, fundus, abdominal
  # CT). octmnist/tissuemnist/chestmnist are large; enable by adding flags.
  medmnist:
    enabled: true
    size: 28
    flags:
      - pathmnist
      - bloodmnist
      - dermamnist
      - pneumoniamnist
      - breastmnist
      - retinamnist
      - organamnist

  # ---- faces / natural images ----
  celeba:          # aligned CelebA; auto-download tried, gdrive quota permitting
    enabled: true
    image_size: 64
  imagenette:      # fast.ai 10-class ImageNet subset; auto-download stand-in
    enabled: true  # for full ImageNet in quick experiments
    variant: 160px
    image_size: 64
  imagenet:        # full ILSVRC2012; manual download (access terms)
    enabled: true
    image_size: 64
  afhq:           # animal faces (cat/dog/wild), StarGAN-v2 release
    enabled: true
    version: v1
    image_size: 64
    attempt_download: true
  ffhq:           # 70k faces; manual download (NVLabs)
    enabled: true
    image_size: 128
  celeba_hq:       # 30k faces; manual download

```

```

enabled: true
image_size: 256
lsun_bedroom: # LSUN lmbd; manual download
enabled: true
image_size: 64

```

2.3 Results: dataset summary

dataset	status	train	val	test	image	classes
mnist	ok	54000	6000	10000	$1 \times 28 \times 28$	10
fashion_mnist	ok	54000	6000	10000	$1 \times 28 \times 28$	10
cifar10	ok	45000	5000	10000	$3 \times 32 \times 32$	10
cifar100	ok	45000	5000	10000	$3 \times 32 \times 32$	100
svhn	ok	65931	7326	26032	$3 \times 32 \times 32$	10
medmnist_pathmnist	ok	89996	10004	7180	$3 \times 28 \times 28$	9
medmnist_bloodmnist	ok	11959	1712	3421	$3 \times 28 \times 28$	8
medmnist_dermamnist	ok	7007	1003	2005	$3 \times 28 \times 28$	7
medmnist_pneumoniamnist	ok	4708	524	624	$1 \times 28 \times 28$	2
medmnist_breastmnist	ok	546	78	156	$1 \times 28 \times 28$	2
medmnist_retinamnist	ok	1080	120	400	$3 \times 28 \times 28$	5
medmnist_organamnist	ok	34561	6491	17778	$1 \times 28 \times 28$	11
celeba	unavailable	—	—	—	—	—
imagenette	ok	8522	947	3925	$3 \times 64 \times 64$	10
imagenet	unavailable	—	—	—	—	—
afhq	ok	13167	1463	1500	$3 \times 64 \times 64$	3
ffhq	unavailable	—	—	—	—	—
celeba_hq	unavailable	—	—	—	—	—
lsun_bedroom	unavailable	—	—	—	—	—

Split sizes are reported after the split policy above; *image* is the tensor shape $C \times H \times W$ delivered by the dataloaders; *classes* is the number of integer classes (“—” for unconditional / multi-label datasets).

2.4 Results: unavailable datasets

- **celeba**: CelebA auto-download failed (Google Drive quota is a known issue: RuntimeError). Manually place `img_align_celeba/` plus the `list_*.txt` / `identity_*` files under `data/celeba/celeba/` (see `torchvision.datasets.CelebA` docs), then re-run.
- **imagenet**: ImageNet (ILSVRC2012) requires manual download (terms of access): obtain from <https://image-net.org>, extract to `data/imagenet/train/<wnid>/*.JPEG` and `data/imagenet/val/<wnid>/*.JPEG`, then re-run.
- **ffhq**: FFHQ requires manual download: place images (e.g. `thumbnails128x128` or `images1024x1024` from github.com/NVlabs/ffhq-dataset) under `data/ffhq/`
- **celeba_hq**: CelebA-HQ requires manual download: place the 30k CelebA-HQ images (e.g. `celeba_hq_256` from the progressive-GAN release) under `data/celeba_hq/`

- **lsun_bedroom**: LSUN bedroom requires manual download: fetch `bedroom_train_lmdb / bedroom_val_lmdb` with `github.com/fyu/lsun download.py` into `data/lsun/` (also needs ‘`pip install lmdb`’).

2.5 Results: figures

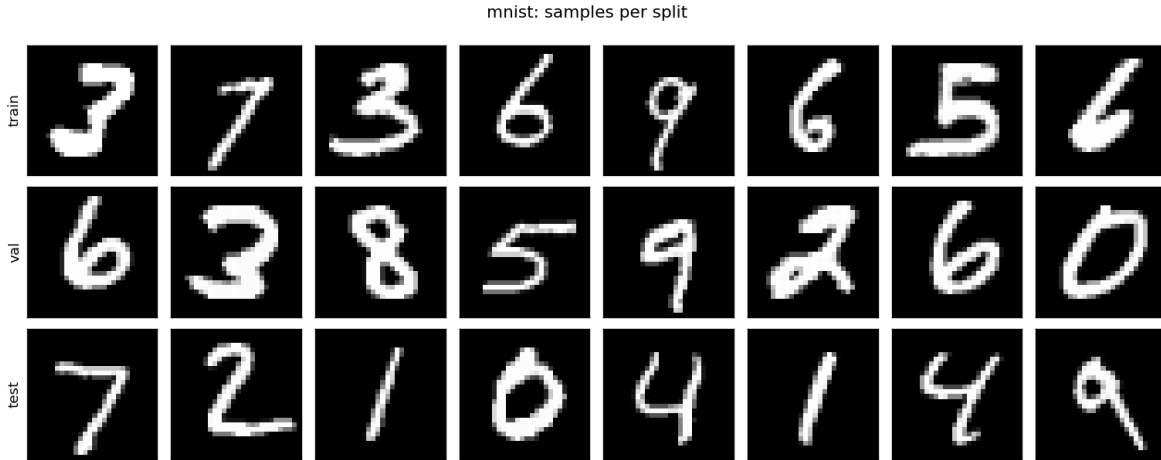


Figure 1: **mnist** (samples). Source: `torchvision.datasets.MNIST` (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/mnist_samples.png`, produced by `step1_datasets.py`.

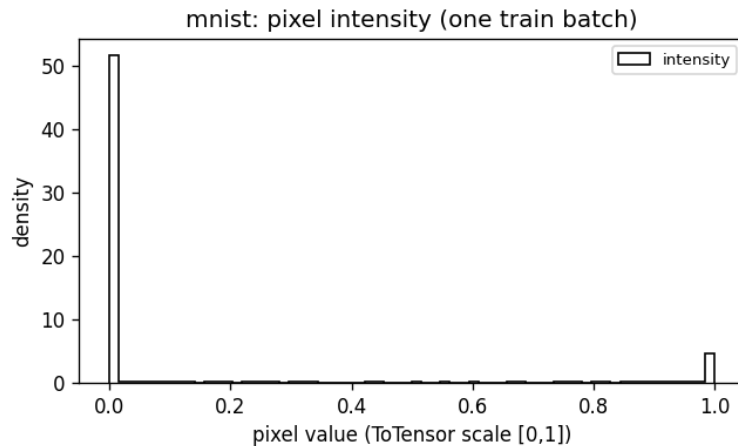


Figure 2: **mnist** (hist). Source: `torchvision.datasets.MNIST` (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/mnist_pixel_hist.png`, produced by `step1_datasets.py`.

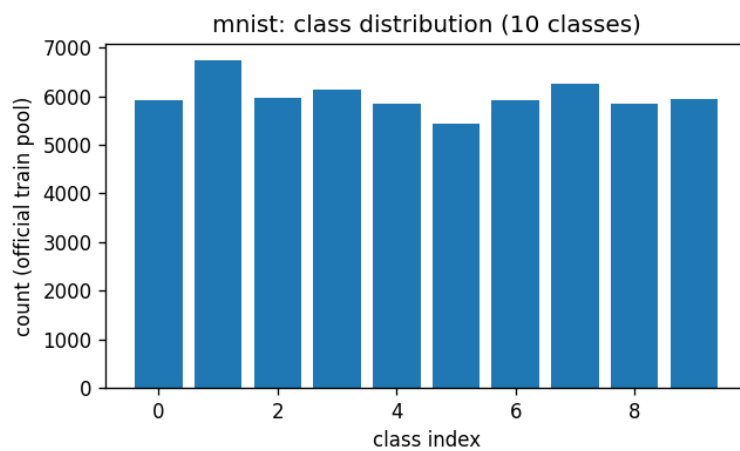


Figure 3: **mnist** (dist). Source: torchvision.datasets.MNIST (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/mnist_class_dist.png, produced by step1_datasets.py.

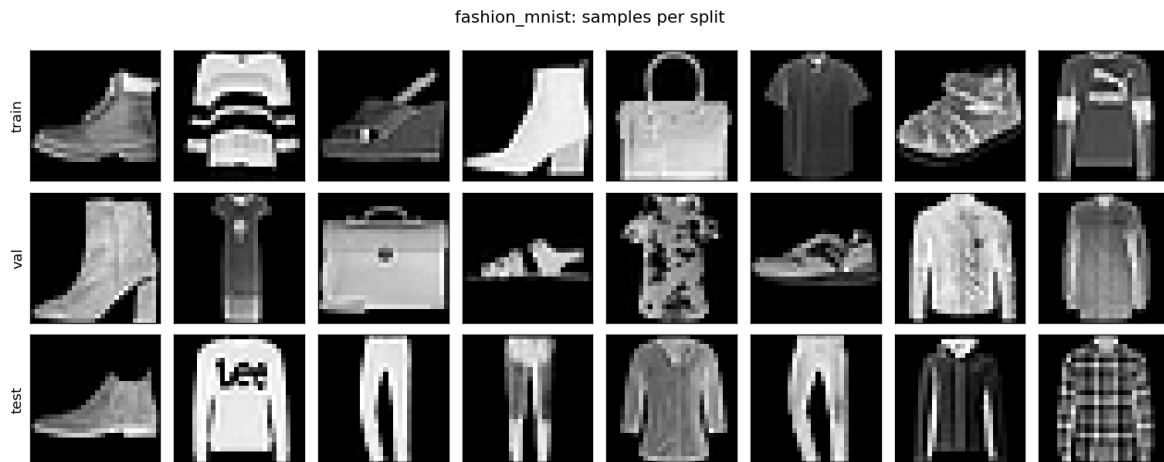


Figure 4: **fashion_mnist** (samples). Source: torchvision.datasets.FashionMNIST (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/fashion_mnist_samples.png, produced by step1_datasets.py.

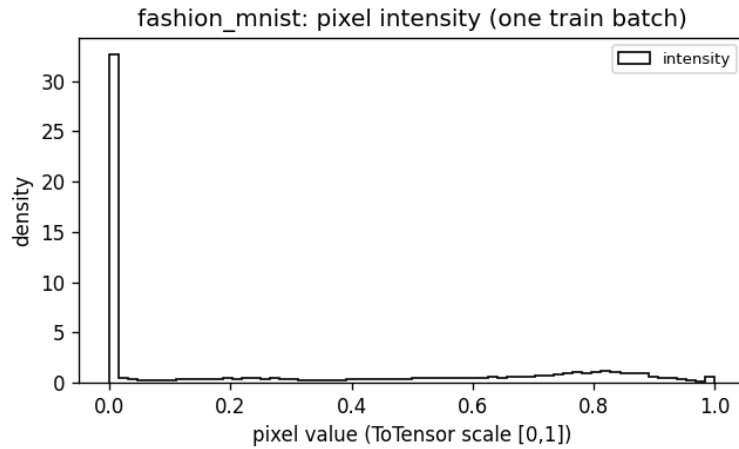


Figure 5: **fashion_mnist** (hist). Source: torchvision.datasets.FashionMNIST (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/fashion_mnist_pixel_hist.png, produced by step1_datasets.py.



Figure 6: **fashion_mnist** (dist). Source: torchvision.datasets.FashionMNIST (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/fashion_mnist_class_dist.png, produced by step1_datasets.py.

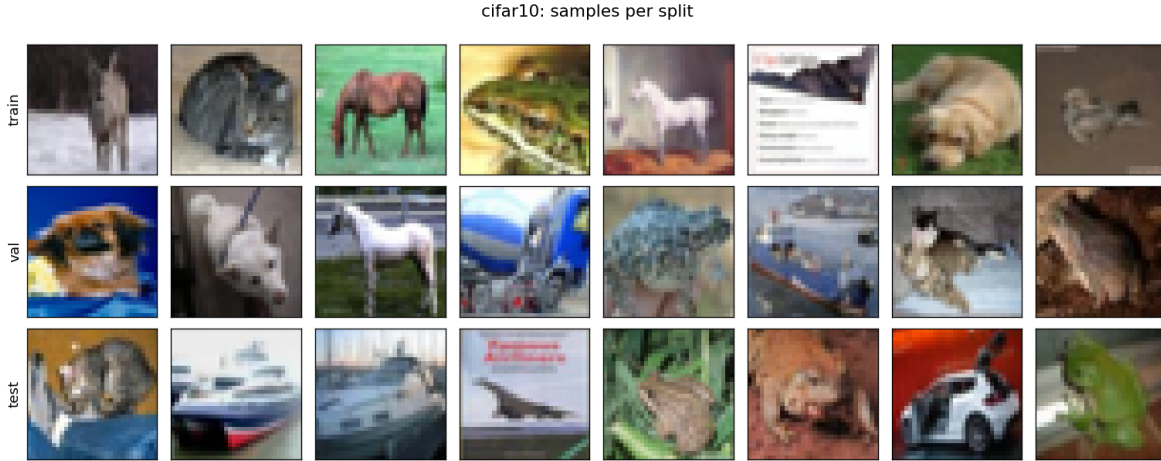


Figure 7: **cifar10** (samples). Source: torchvision.datasets.CIFAR10 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/cifar10_samples.png, produced by step1_datasets.py.

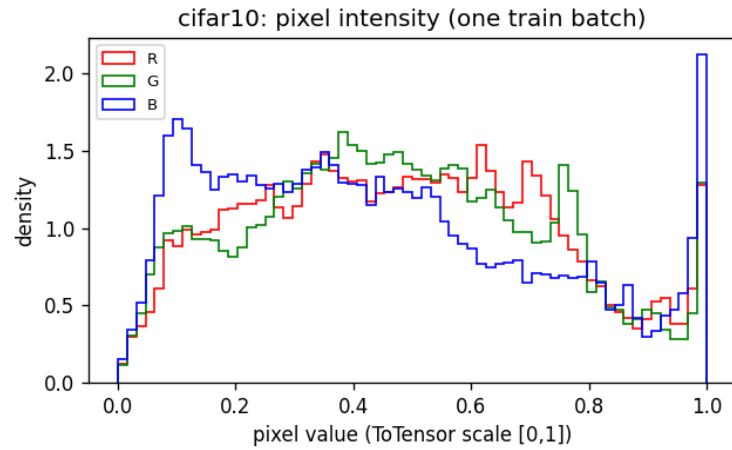


Figure 8: **cifar10** (hist). Source: torchvision.datasets.CIFAR10 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/cifar10_pixel_hist.png, produced by step1_datasets.py.

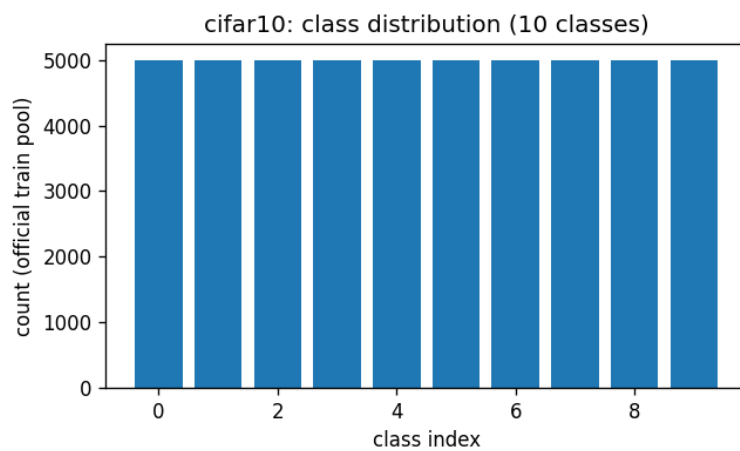


Figure 9: **cifar10** (dist). Source: torchvision.datasets.CIFAR10 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/cifar10_class_dist.png, produced by step1_datasets.py.

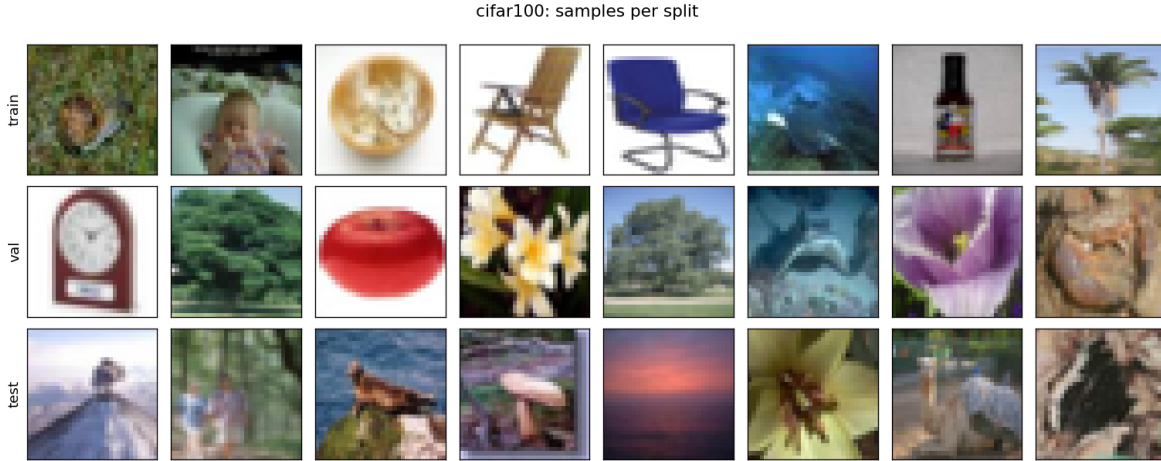


Figure 10: **cifar100** (samples). Source: torchvision.datasets.CIFAR100 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/cifar100_samples.png, produced by step1_datasets.py.

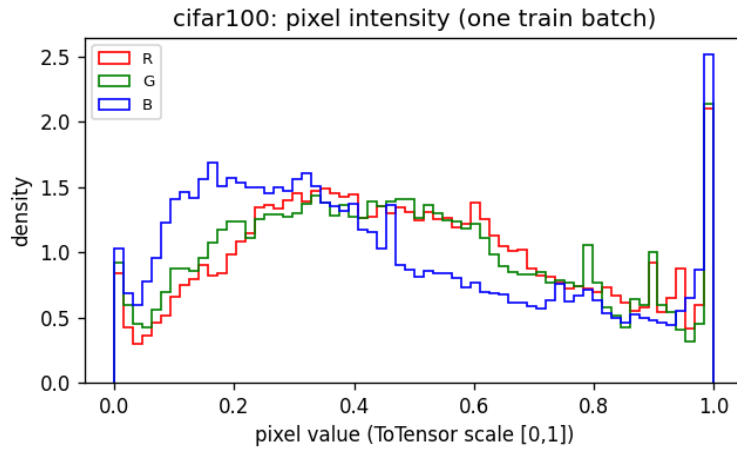


Figure 11: **cifar100** (hist). Source: torchvision.datasets.CIFAR100 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/cifar100_pixel_hist.png, produced by step1_datasets.py.

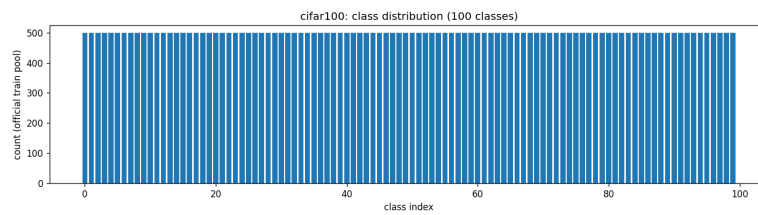


Figure 12: **cifar100** (dist). Source: torchvision.datasets.CIFAR100 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/cifar100_class_dist.png, produced by step1_datasets.py.

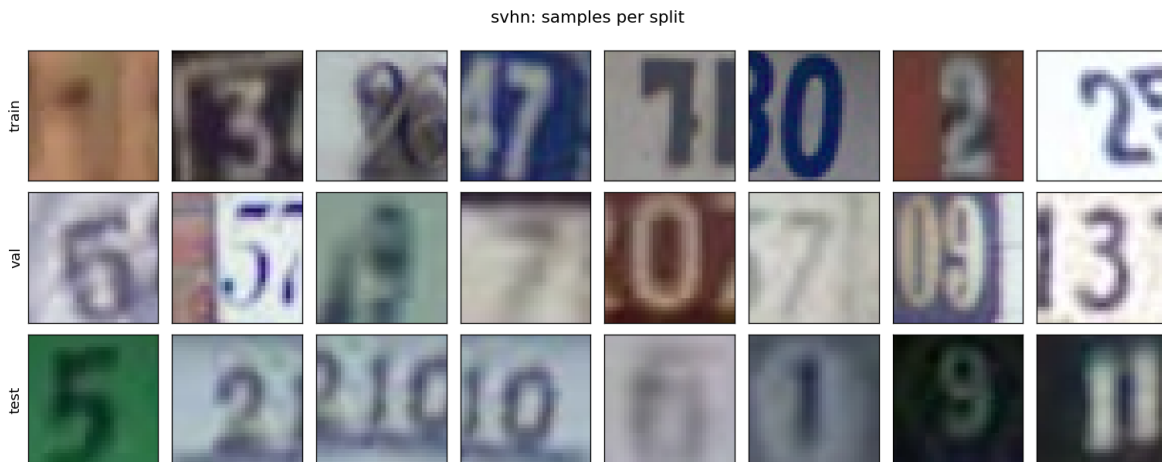


Figure 13: **svhn** (samples). Source: torchvision.datasets.SVHN (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/svhn_samples.png, produced by step1_datasets.py.

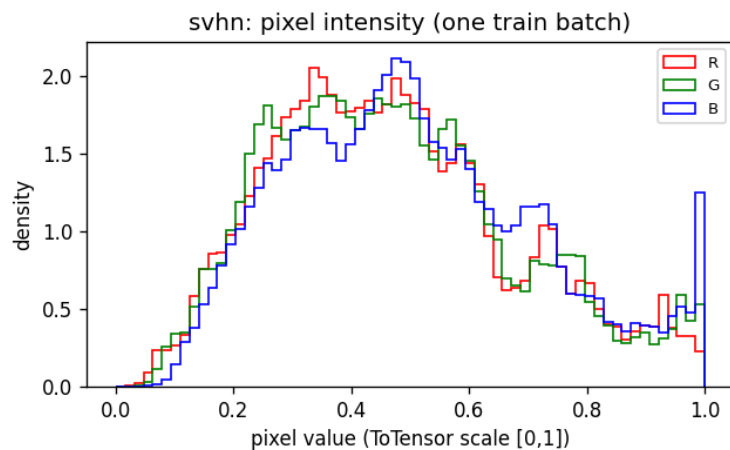


Figure 14: **svhn** (hist). Source: torchvision.datasets.SVHN (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/svhn_pixel_hist.png, produced by step1_datasets.py.

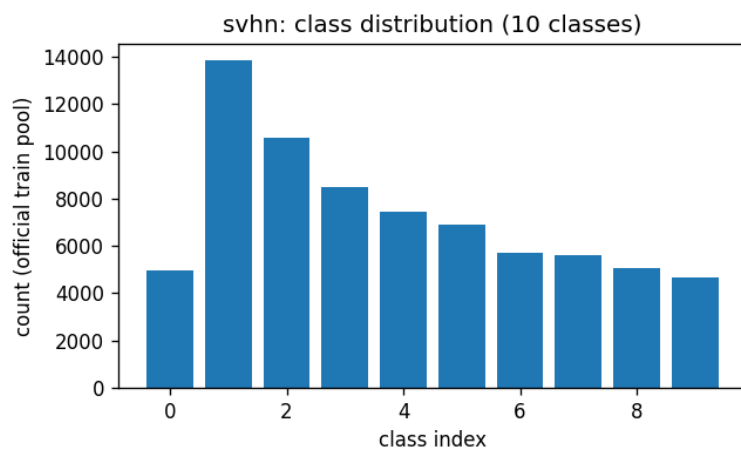


Figure 15: **svhn** (dist). Source: torchvision.datasets.SVHN (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/svhn_class_dist.png, produced by step1_datasets.py.

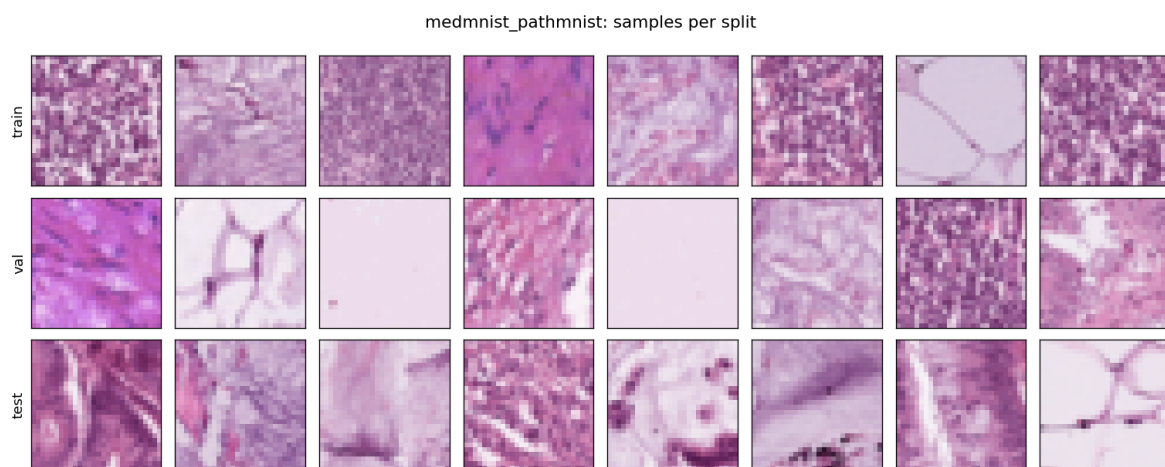


Figure 16: **medmnist_pathmnist** (samples). Source: medmnist.PathMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_pathmnist_samples.png, produced by step1_datasets.py.

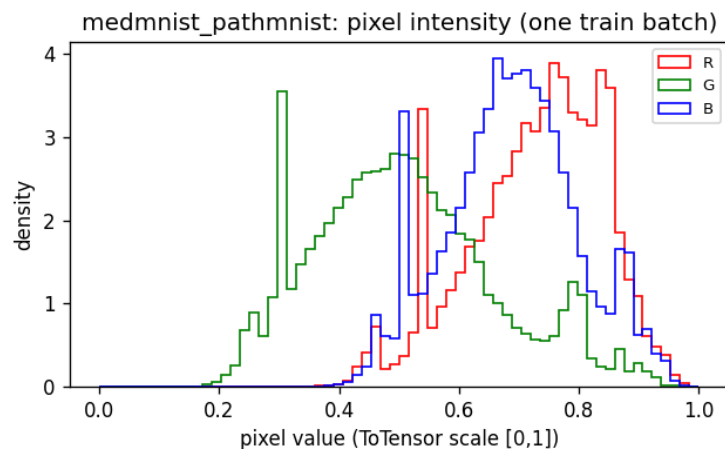


Figure 17: **medmnist_pathmnist** (hist). Source: medmnist.PathMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_pathmnist_pixel_hist.png, produced by step1_datasets.py.

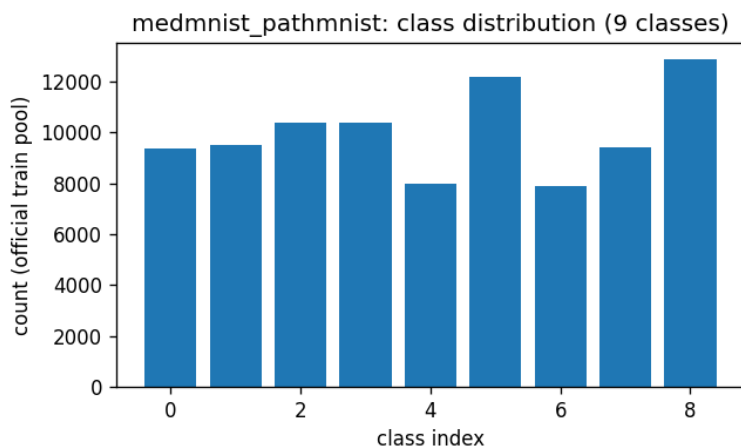


Figure 18: **medmnist_pathmnist** (dist). Source: medmnist.PathMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_pathmnist_class_dist.png, produced by step1_datasets.py.

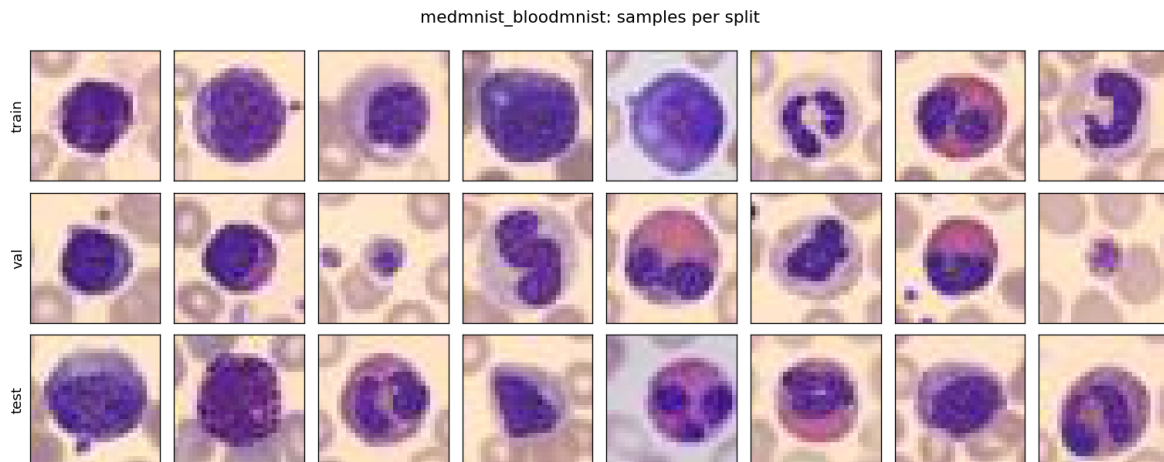


Figure 19: **medmnist_bloodmnist** (samples). Source: medmnist.BloodMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_bloodmnist_samples.png, produced by step1_datasets.py.

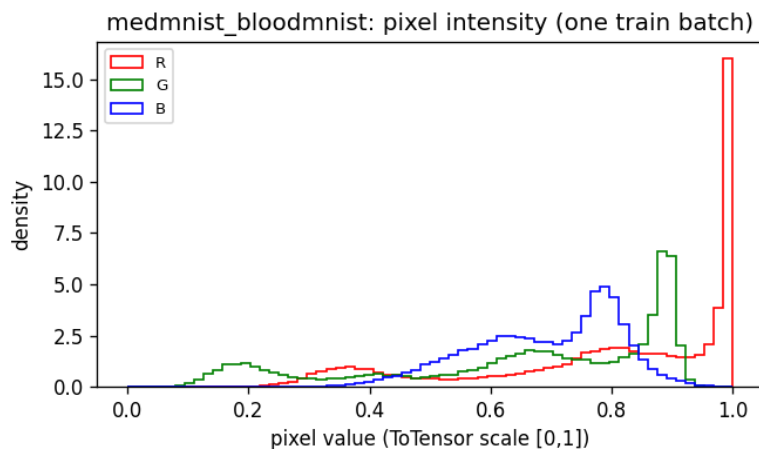


Figure 20: **medmnist_bloodmnist** (hist). Source: medmnist.BloodMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_bloodmnist_pixel_hist.png, produced by step1_datasets.py.

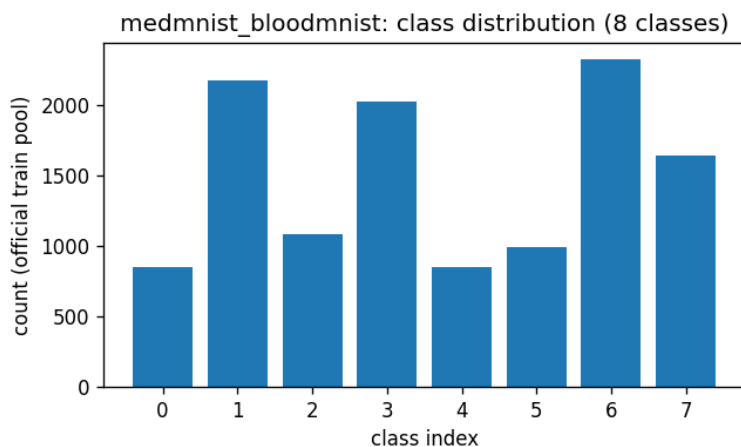


Figure 21: **medmnist_bloodmnist** (dist). Source: medmnist.BloodMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_bloodmnist_class_dist.png, produced by step1_datasets.py.

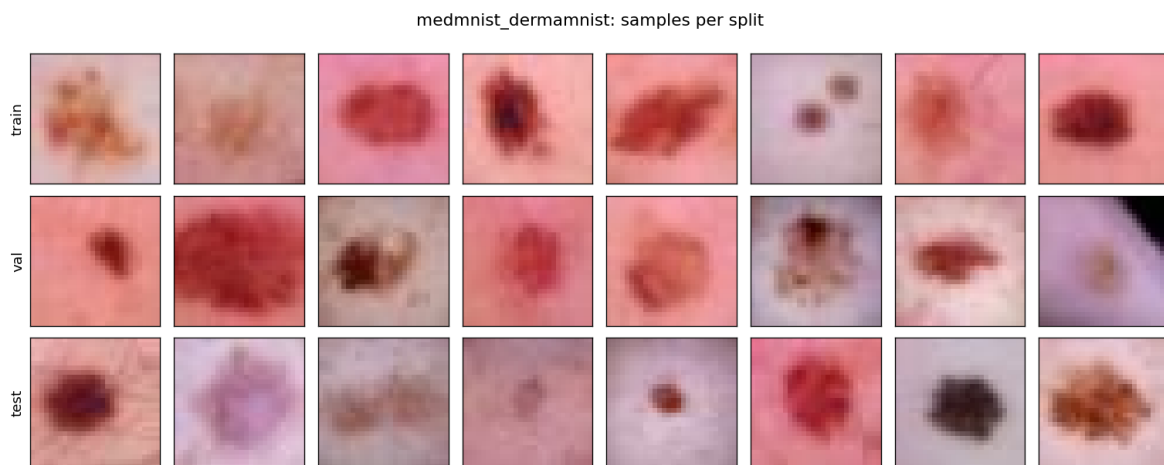


Figure 22: **medmnist_dermamnist** (samples). Source: medmnist.DermaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_dermamnist_samples.png, produced by step1_datasets.py.

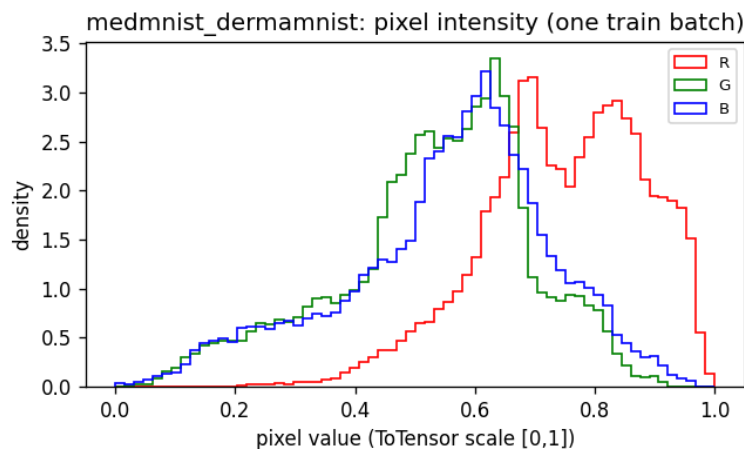


Figure 23: **medmnist_dermamnist** (hist). Source: medmnist.DermaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_dermamnist_pixel_hist.png, produced by step1_datasets.py.

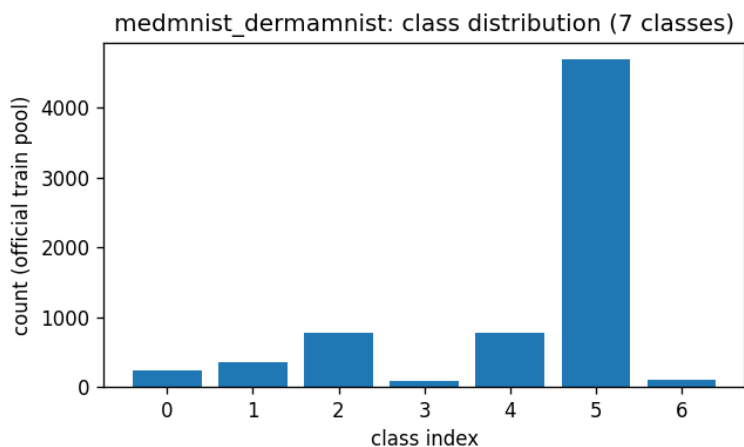


Figure 24: **medmnist_dermamnist** (dist). Source: medmnist.DermaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_dermamnist_class_dist.png, produced by step1_datasets.py.

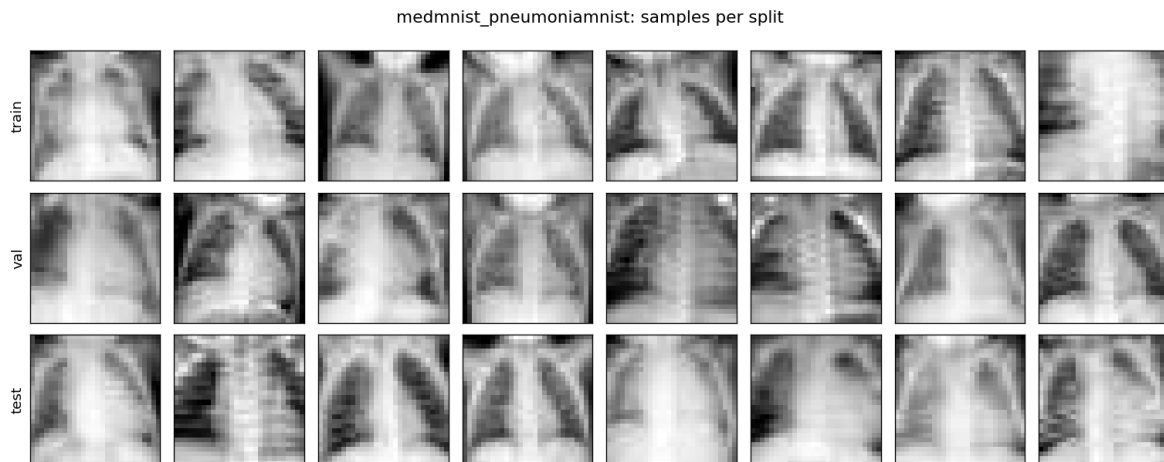


Figure 25: **medmnist_pneumoniamnist** (samples). Source: `medmnist.PneumoniaMNIST` v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_pneumoniamnist_samples.png`, produced by `step1_datasets.py`.

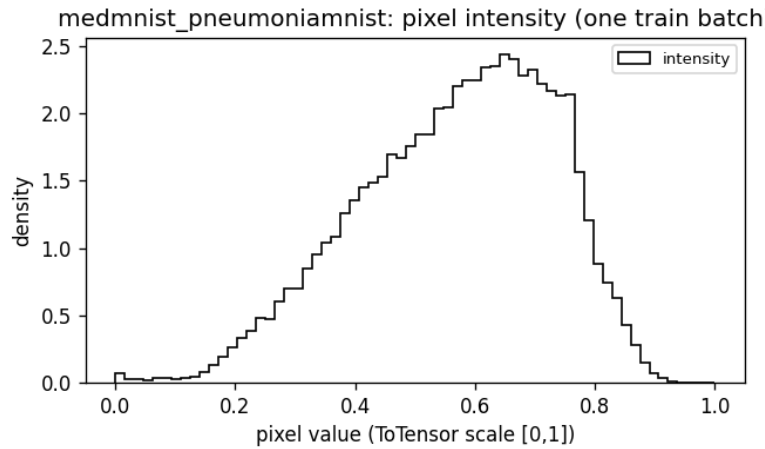


Figure 26: **medmnist_pneumoniamnist** (hist). Source: medmnist.PneumoniaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_pneumoniamnist_pixel_hist.png, produced by step1_datasets.py.

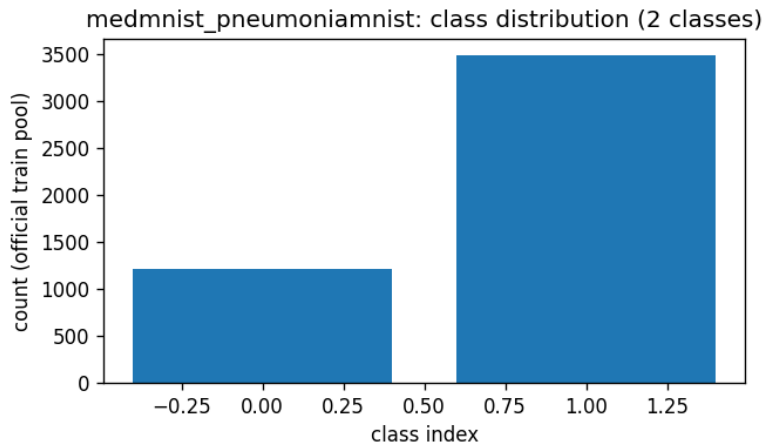


Figure 27: **medmnist_pneumoniamnist** (dist). Source: medmnist.PneumoniaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_pneumoniamnist_class_dist.png, produced by step1_datasets.py.

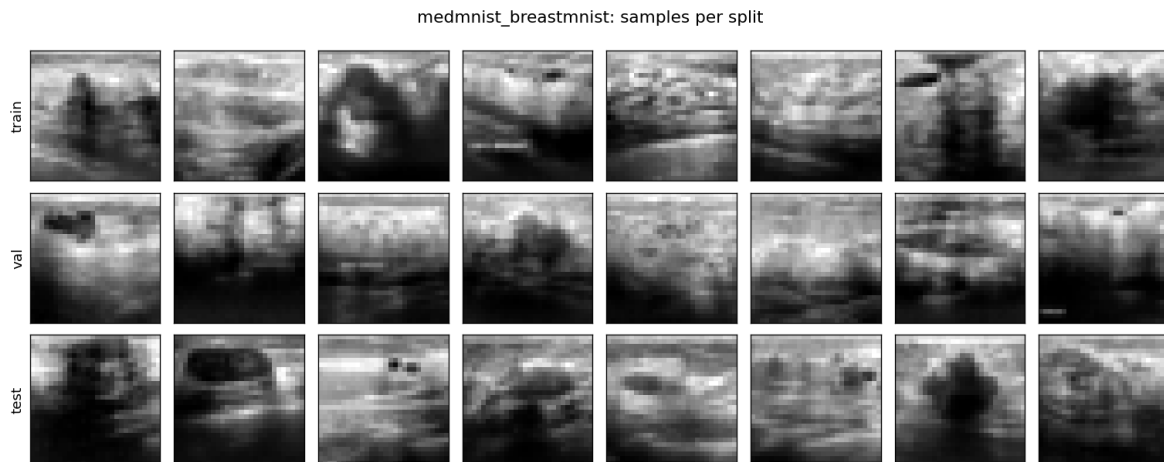


Figure 28: **medmnist_breastmnist** (samples). Source: medmnist.BreastMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_breastmnist_samples.png, produced by step1_datasets.py.

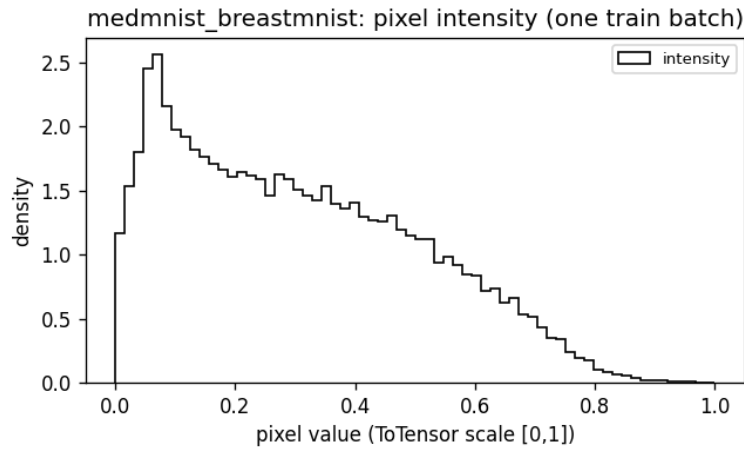


Figure 29: **medmnist_breastmnist** (hist). Source: medmnist.BreastMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_breastmnist_pixel_hist.png, produced by step1_datasets.py.

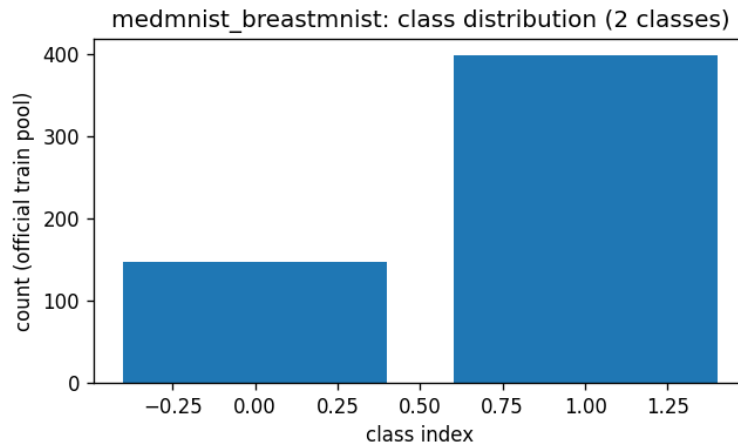


Figure 30: **medmnist_breastmnist** (dist). Source: medmnist.BreastMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_breastmnist_class_dist.png, produced by step1_datasets.py.

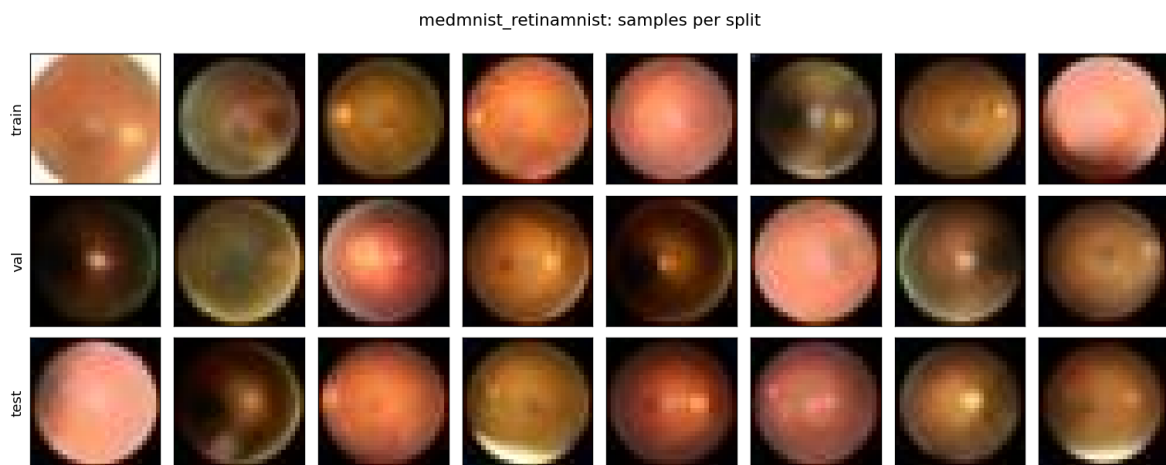


Figure 31: **medmnist_retinamnist** (samples). Source: medmnist.RetinaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_retinamnist_samples.png, produced by step1_datasets.py.

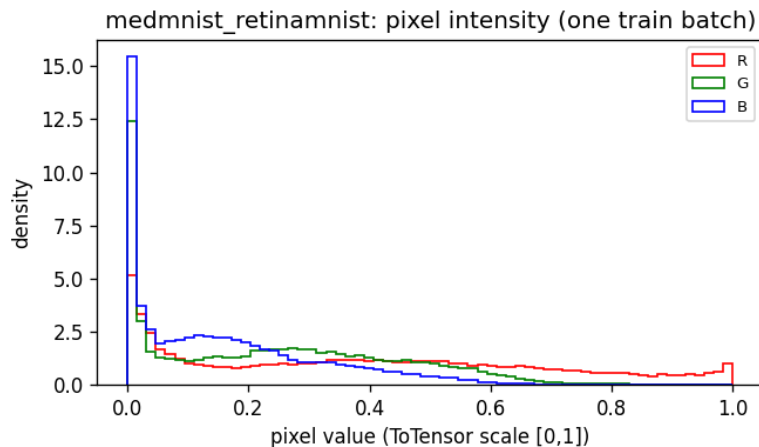


Figure 32: **medmnist_retinamnist** (hist). Source: medmnist.RetinaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_retinamnist_pixel_hist.png, produced by step1_datasets.py.

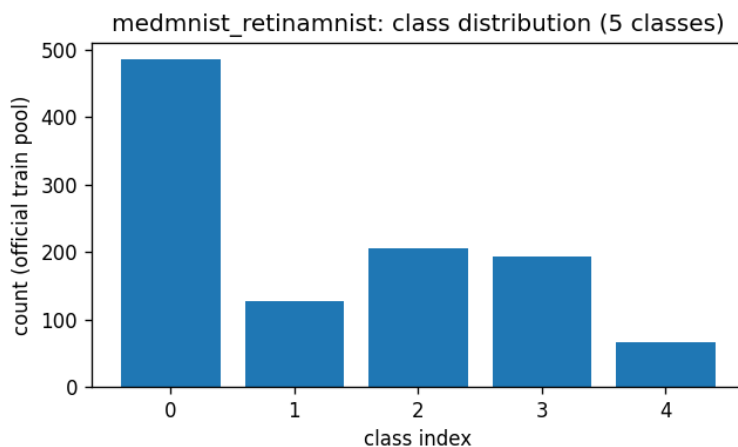


Figure 33: **medmnist_retinamnist** (dist). Source: medmnist.RetinaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_retinamnist_class_dist.png, produced by step1_datasets.py.

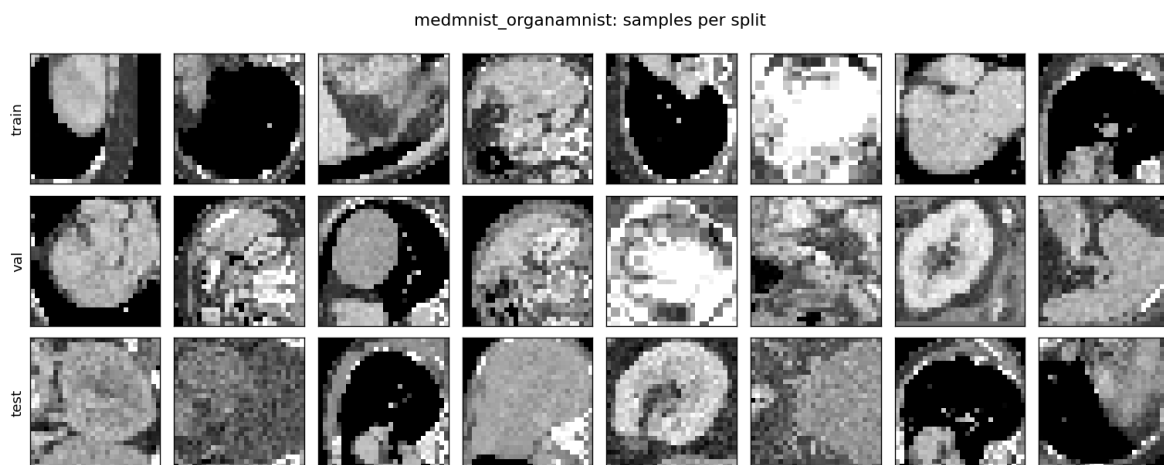


Figure 34: **medmnist_organamnist** (samples). Source: medmnist.OrganAMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_organamnist_samples.png, produced by step1_datasets.py.

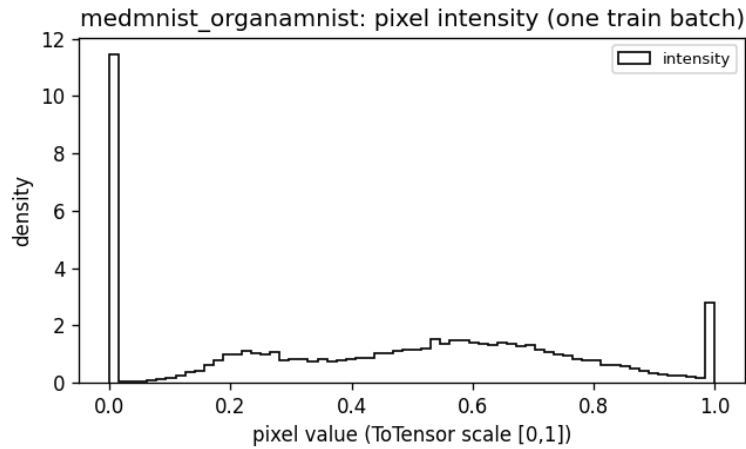


Figure 35: **medmnist_organamnist** (hist). Source: medmnist.OrganAMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_organamnist-pixel_hist.png, produced by step1_datasets.py.

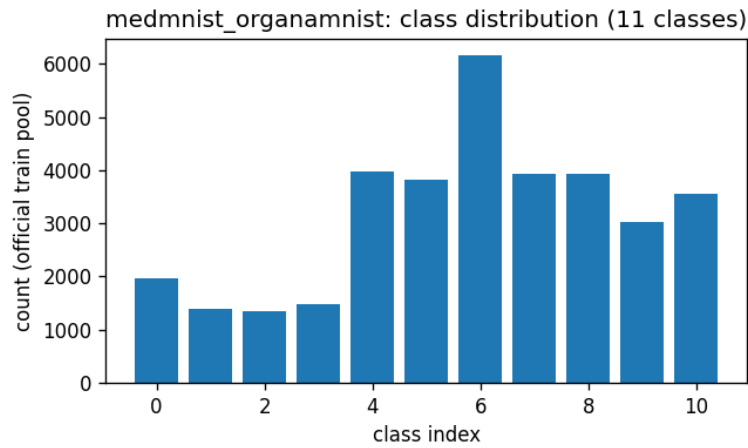


Figure 36: **medmnist_organamnist** (dist). Source: medmnist.OrganAMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_organamnist-class_dist.png, produced by step1_datasets.py.

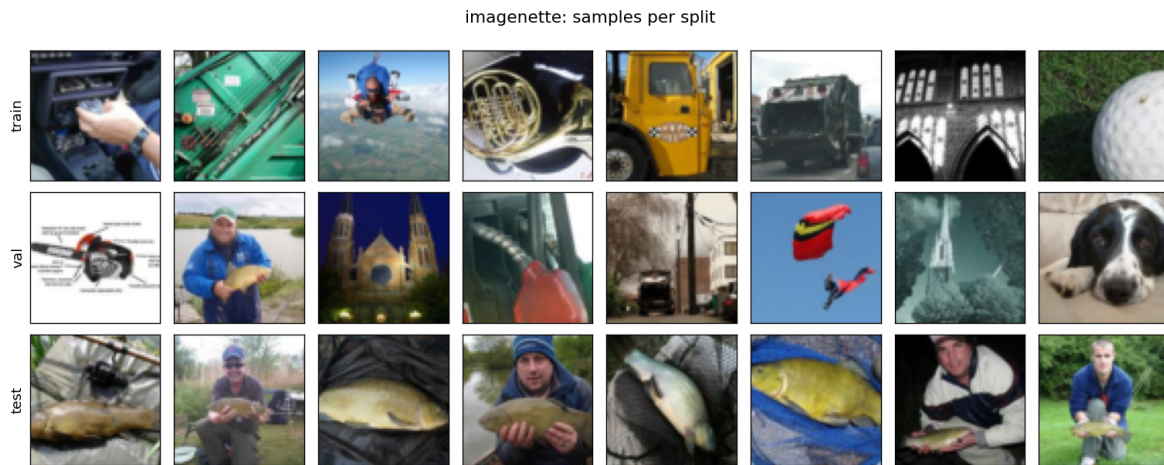


Figure 37: **imagenette** (samples). Source: `torchvision.datasets.Imagenette` 160px (fast.ai ImageNet-10 subset, auto-download). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/imagenette_samples.png`, produced by `step1_datasets.py`.

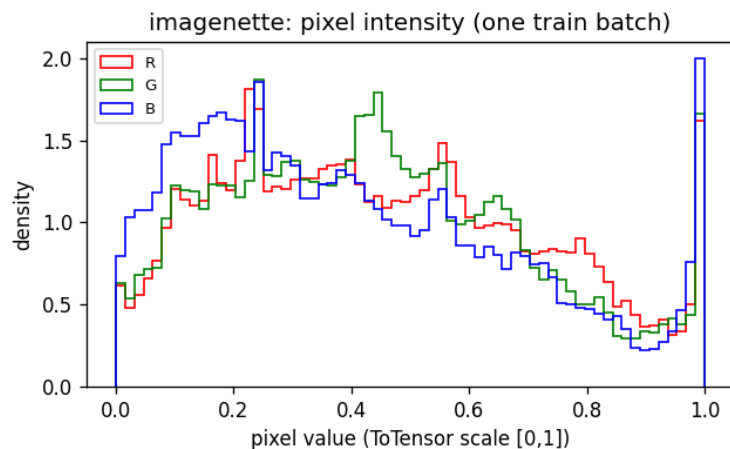


Figure 38: **imagenette** (hist). Source: torchvision.datasets.Imagenette 160px (fast.ai ImageNet-10 subset, auto-download). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/imagenette_pixel_hist.png, produced by step1_datasets.py.

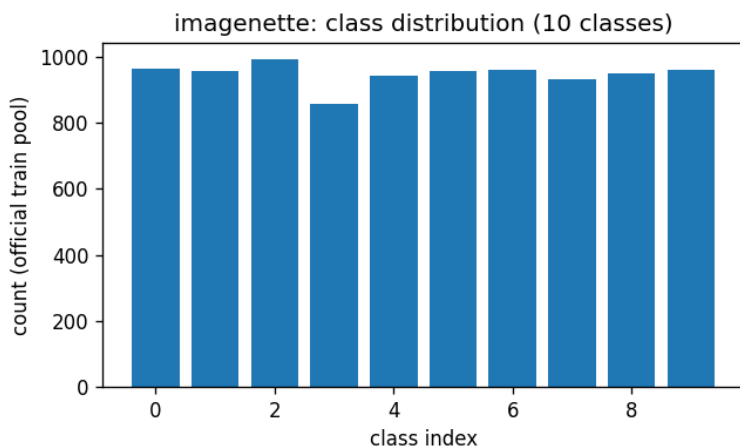


Figure 39: **imagenette** (dist). Source: torchvision.datasets.Imagenette 160px (fast.ai ImageNet-10 subset, auto-download). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/imagenette_class_dist.png, produced by step1_datasets.py.



Figure 40: **afhq** (samples). Source: AFHQ v1 (StarGAN-v2 release, cat/dog/wild, auto-download from dropbox). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/afhq_samples.png`, produced by `step1_datasets.py`.

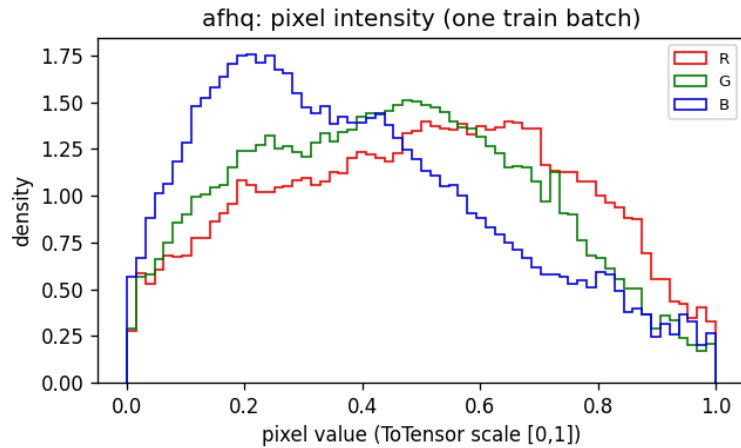


Figure 41: **afhq** (hist). Source: AFHQ v1 (StarGAN-v2 release, cat/dog/wild, auto-download from dropbox). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/afhq_pixel_hist.png`, produced by `step1_datasets.py`.

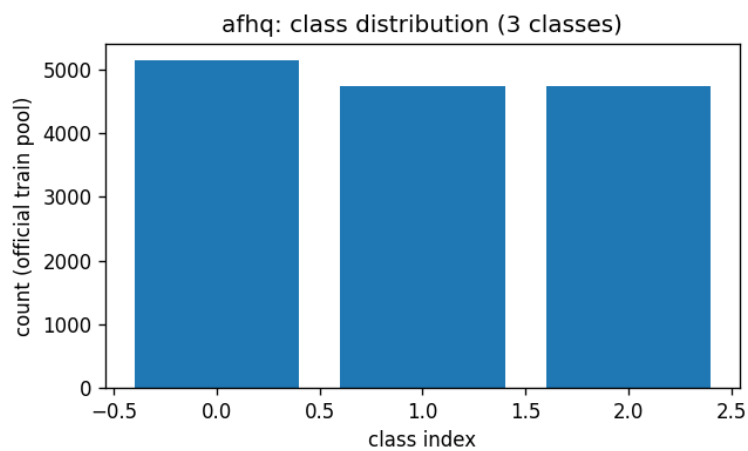


Figure 42: **afhq** (dist). Source: AFHQ v1 (StarGAN-v2 release, cat/dog/wild, auto-download from dropbox). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/afhq_class_dist.png, produced by step1_datasets.py.

2.6 Analysis

- Coverage: the available benchmarks span 28×28 grayscale (MNIST, several MedMNIST members) through 64×64 RGB natural images and faces (CIFAR, Imagenette, AFHQ, CelebA), which is the standard ladder for likelihood-based generative models: density results in bits/dim on MNIST/CIFAR-10, then perceptual-quality results on faces/animals at higher resolution.
- The pixel-intensity histograms show the strongly peaked, saturated distributions typical of 8-bit images (large masses at 0 and 1, e.g. MNIST background, CelebA skin tones). For normalizing flows this directly motivates uniform dequantization and a logit transform before the first coupling layer – to be implemented and ablated in the model step, not in the data step.
- Class-distribution figures document which datasets are usable for class-conditional modeling and how imbalanced they are; several MedMNIST members are markedly imbalanced, which matters for conditional likelihood evaluation.
- Per-channel means/stds recorded in each dataset’s JSON give the constants for any input normalization later steps choose to apply.
- Datasets marked unavailable (typically ImageNet, FFHQ, CelebA-HQ, LSUN, and CelebA when the Google Drive quota blocks download) are access-restricted or manually hosted; the pipeline documents the exact acquisition path and picks them up automatically once placed under `/data/image_benchmarks`.

3 Reproducibility

- Global seed 20260831 (config) seeds torch, numpy, and every `random_split`; splits are therefore identical across runs and machines.
- `outputs/` and dataset caches are gitignored; the committed code + config + this PDF are the record, and any output can be regenerated with the commands above.
- Provenance for the run in this PDF:

```
{
  "step": "step1_datasets",
  "command": "python step1_datasets.py",
  "config_file": "/dev_ws/invertible_normalizing_flow_20260831/configs/step1_datasets.yml",
  "config": {
    "seed": 20260831,
    "data_root": "/data/image_benchmarks",
    "output_root": "outputs/step1_datasets",
    "loader": {
      "batch_size": 64,
      "num_workers": 4,
      "pin_memory": true
    },
    "split": {
      "val_fraction": 0.1,
      "test_fraction": 0.1
    },
    "figures": {
      "samples_per_split": 8,
      "stats_max_batches": 20
    }
  }
}
```

```

},
"datasets": {
  "mnist": {
    "enabled": true
  },
  "fashion_mnist": {
    "enabled": true
  },
  "cifar10": {
    "enabled": true
  },
  "cifar100": {
    "enabled": true
  },
  "svhn": {
    "enabled": true
  },
  "medmnist": {
    "enabled": true,
    "size": 28,
    "flags": [
      "pathmnist",
      "bloodmnist",
      "dermamnist",
      "pneumoniamnist",
      "breastmnist",
      "retinamnist",
      "organamnist"
    ]
  },
  "celeba": {
    "enabled": true,
    "image_size": 64
  },
  "imagenette": {
    "enabled": true,
    "variant": "160px",
    "image_size": 64
  },
  "imagenet": {
    "enabled": true,
    "image_size": 64
  },
  "afhq": {
    "enabled": true,
    "version": "v1",
    "image_size": 64,
    "attempt_download": true
  },
  "ffhq": {
    "enabled": true,
    "image_size": 128
  },
  "celeba_hq": {
    "enabled": true,
    "image_size": 256
  },
  "lsun_bedroom": {
    "enabled": true,
    "image_size": 64
  }
}
},
"timestamp_utc": "2026-08-31T21:22:10.697662+00:00",
"versions": {
  "python": "3.10.12",
  "torch": "2.11.0+cu128",
  "torchvision": "0.26.0+cu128",

```

```
    "numpy": "2.2.6",  
    "medmnist": "3.0.2"  
  },  
  "cuda_available": true  
}
```